

03、SDK说明

SDK例程说明

一、介绍

这是一个双足机器人控制和通信的软件接口包。

通过这个接口包，可以方便地与双足机器人底层硬件通信，完成状态信息获取和控制电机动作。

二、结构

程序主体在src文件夹下，我们主要使用文件为livelybot_bringup、livelybot_description、livelybot_serial。

代码块

```
1  src/
2  ├── livelybot_bringup/      # 启动与管理脚本、配置、launch文件
3  ├── livelybot_description/  # 机器人模型与参数描述（如参数yaml等）
4  ├── livelybot_logger/      # 日志记录与服务
5  ├── livelybot_msg/         # ROS消息定义
6  ├── livelybot_oled/        # OLED显示相关节点
7  ├── livelybot_serial/      # 串口通信与底层硬件SDK（核心控制）
8  ├── serial/                # 通用串口库
9  ├── yesense_ros/           # IMU等传感器相关驱动
```

控制电机使用的是livelybot_serial的motor中的控制函数，在本文6.1.4中讲解。

三、配置

livelybot_description文件夹下是电机参数描述文件，对该文件下的描述文件进行修改

3.1修改livelybot_description.launch文件

1.修改 `dof_type` 的数量，根据单个 CAN 通道需连接的最大电机数量修改。我们提供1、 2、 6、 12、 20、 23、 80的电机配置。假设需要使用2个 CAN 通道，其中 CAN1 接 1 个电机，CAN2 接 3 个电机，宏定义 `dof_type` 修改为 6。

代码块

```
1  <arg name = "dof_type" default = "6"/>
```

3.2修改robot_param中的yaml配置文件

以1dof_STM32H730_model_test_Orin_params为例

1. 机器人根节点

- 配置文件以robot:开始;

2. 软硬件参数说明

- `SDK_version: 2` 表示使用的SDK软件版本为2.0;
- `robot_name: "mini pai"` 表示机器人的名字为 `mini pai`;
- `arm_dof:0` 表示机器人的手臂自由度为0, `leg_dof: 6`表示机器人的腿部自由度为6;
- `Serial_Type: "/dev/ttyACM"` 表示机器人通信板的串口名称为 `/dev/ttyACM*`;
- `Seial_baudrate:4000000` 表示机器人通信板的串口波特率为4000000;
- `CAN_TYPE: "CAN-FD"` 表示机器人的CAN总线类型为 `CAN-FD` , 可选项有: `"CAN-FD"`、`"CAN 2.0B"`;
- `control_type: 9` 表示机器人的控制模式为9, 即运控模式, 此处选项对应 `motor` 中的 `fresh_cmd_int16` 函数, 若直接调用 `motor` 的控制函数则与该选项无关。

■ 可选项有:

- 1 : 位置模式
- 2 : 速度模式
- 3 : 力矩模式
- 4 : 电压模式
- 5 : 电流模式
- 6 : 位置-速度-最大力矩模式
- 7 : 弃用
- 8 : 弃用
- 9 : 运控模式
- 10 : 带 Kp-Kd 参数的位置-速度模式
- 11 : 位置-速度-加速度 (梯形控制) 模式
- 12 : 运控模式 2

■ 例:

代码块

```
1 #1.调用函数,此时配置文件中为9,即运控模式
```

```

2  motor.fresh_cmd_int16 (...)
3  robot.motor_send_2()
4
5  #2.直接调用控制函数
6  #位置模式
7  motor.position(position)
8  robot.motor_send_2()
9
10 #速度模式
11 motor.velocity(velocity)
12 robot.motor_send_2()

```

- `CANboard_num: 1` 表示机器人通信板个数为1;

3. 通信板配置信息

- `CANboard`: 表示机器人的通信板信息，包含CAN总线数和每条CAN总线上的电机数;
- `No_1_CANboard`: 表示第一块通信板的节点;
- `CANport_1`: 表示第一条CAN总线上的电机信息;
- `motor_num: 1` 表示这条CAN线上的电机个数;

4. 电机配置信息

- `motor_1`: 表示这条CAN线上的第1个电机信息;
- `type:4438_30`: 电机类型，可选项有: `5046电机-20减速比`、`4538电机-20 减速比`、`5047电机-36 减速比`、`5047电机-9减速比`、`4438 电机-32减速比`、`4438电机, 8减速比`、`7136电机-9减速比` ;
- `id: 1`: 电机ID
- `name: "L_low_foot"`: 电机名称;
- `pos_limit_enable: false`: 位置保护使能标识, `true` 表示使能, `false` 表示不使
- `pos_upper: 5`: 位置上限, 单位是圈数;
- `pos_lower: -5`: 速度下限, 单位是圈数;
- `tor_limit_enable: false`: 力矩限制使能标识, `true` 表示使能, `false` 表示不使
- `tor_upper: 5`: 力矩上限, 单位是Nm;
- `tor_lower: -3`: 力矩下限, 单位是Nm;

四、运行流程

程序运行是先读取livelybot_description中的配置文件，如下图所示，

```
hightorque@lubancat: ~/SDK_v4.4.5.6_250807/livelybot_hard...
SUMMARY
=====
PARAMETERS
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor1/id: 1
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor1/name: L_low_foot
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor1/num: 1
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor1/pos_limit_enable: False
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor1/pos_lower: -5
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor1/pos_upper: 5
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor1/tor_limit_enable: False
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor1/tor_lower: -3
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor1/tor_upper: 5
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor1/type: 4538_19
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor2/id: 2
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor2/name: L_up_foot
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor2/num: 2
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor2/pos_limit_enable: False
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor2/pos_lower: -5
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor2/pos_upper: 5
```

```
hightorque@lubancat: ~/SDK_v4.4.5.6_250807/livelybot_hard...
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor2/tor_limit_enable: False
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor2/tor_lower: -3
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor2/tor_upper: 5
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor/motor2/type: 4538_19
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/motor_num: 1
* /robot/CANboard/No_1_CANboard/CANport/CANport_1/serial_id: 1
* /robot/CANboard_num: 1
* /robot/SDK_version: 2
* /robot/Serial_baudrate: 4000000
* /robot/Serial_Type: /dev/ttyACM
* /robot/arm_dof: 0
* /robot/control_type: 9
* /robot/imu_dir: False
* /robot/imu_limit_flag: False
* /robot/imu_limit_num: 1.0
* /robot/leg_dof: 6
* /robot/robot_name: mini pai
* /rostdistro: noetic
* /rosverison: 1.17.3

NODES
/
```

之后运行robot的初始化程序，读取sdk版本、机器人名字、功率板数量、串口信息和电机信息，之后就是运行示例程序如下图所示。

SDK_version为sdk版本

Motor version为电机固件版本

```
hightorque@lubancat: ~/SDK_v4.4.5.6_250807/livelybot_hard...
process[motor_rerun-1]: started with pid [12997]
[INFO] [1754962955.755121821]: Got params SDK_version: v4.4.5
[INFO] [1754962955.759077825]: The robot name is mini pai
[INFO] [1754962955.759170856]: The robot has 1 CANboards
[INFO] [1754962955.759253679]: The Serial type is /dev/ttyACM
Serial Port List:
[INFO] [1754962955.760559904]: Serial Port0 = /dev/ttyACM0
[INFO] [1754962955.761127422]: Serial Port1 = /dev/ttyACM1
[INFO] [1754962955.761667527]: Serial Port2 = /dev/ttyACM2
[INFO] [1754962955.762212589]: Serial Port3 = /dev/ttyACM3
[INFO] [1754962955.764517074]: board 1 has 1 port
[INFO] [1754962955.779062400]: Got params type: 4538_19
[INFO] [1754962955.809605553]: CANboard(1) version is: v4.0
[INFO] [1754962955.809741454]: Detecting motor connection
[INFO] [1754962955.910183440]: All motor connections are normal
[INFO] [1754962955.910401873]: -----motor version-----
[INFO] [1754962955.910515610]: motors[00]: id:01 v4.5.2
[INFO] [1754962955.910637804]: -----
[INFO] [1754962955.931246350]: fun_v = 5
[INFO] [1754962956.035995858]: The robot has 1 motors
[INFO] [1754962956.036200585]: robot init
[INFO] [1754962956.036298574]: ALL Motor: 1
```

下图为获取版本程序编写

SDK版本获取

电机固件版本获取

```
robot::robot()
{
    if (n.getParam("robot/SDK_version", SDK_version))
    {
        // ROS_INFO("Got params SDK_version: %f", SDK_version);
    }
    else
    {
        ROS_ERROR("Faile to get params SDK_version");
        SDK_version = -1;
    }
    if (n.getParam("robot/Seial_baudrate", Seial_baudrate))
    {
        // ROS_INFO("Got params seial_baudrate: %s", seial_baudrate.c_str());
    }
}
```

```
set_port_motor_num(); // 设置通道上挂载的电机数，并获取主控板固件版本号
if (slave_v >= 4.1f)
{
    canboard_fdcan_reset();
}

if (slave_v < 4.0f) // 检测电机连接是否正常
{
    fun_v = fun_v1;
    chevk_motor_connection_position();
}
else
{
    fun_v = fun_v2;
    chevk_motor_connection_version();
}
```

五、测试例程

5.1、motor_rerun

该程序是一个基本的电机控制示例，适用于测试电机的运行、发送控制命令和接收电机状态信息。该程序控制前半部分电机控制正向，后半部分控制反向，并每 250 次循环反转一次运动方向。

5.2、motor_set_zero

这个程序的功能是设置电机当前位置为零点并周期性地读取并打印每个电机的状态。

5.3、motor_move_zero

这个程序的功能是控制电机回到零点位置并周期性地读取并打印每个电机的状态。

5.4、motor_timeout

这个程序的功能是设置电机的超时时间，控制多个电机并定期发送速度指令，同时接收并打印电机的状态反馈信息（如位置、速度、扭矩）。

5.5、motor_feedback

这个程序的功能是通过 ROS 监控多个电机，并定期获取和打印电机的状态反馈信息（如位置、速度、扭矩）。它与电机进行通信并循环读取其状态，以便实时查看电机的工作状态。

5.6、canboard_update

该程序的功能是通过 ROS 节点调用一个方法 `rb.canboard_bootloader()` 来让控制板进入 **Bootloader 模式**

5.7、测试程序解析

5.7.1 电机旋转测试motor_rerun

- 头文件

代码块

```
1  #include "ros/ros.h"
2  #ifndef RELEASE
```

```

3  #include "robot.h"
4  #else
5  #include "livelybot_serial/hardware/robot.h"
6  #endif
7  #include <iostream>
8  #include <thread>
9  #include <condition_variable>

```

- ROS 初始化与设置

代码块

```

1  ros::init(argc, argv, "test_motor");
2  ros::NodeHandle n;
3  ros::Rate r(200);
4  livelybot_serial::robot rb;

```

`ros::init`: 初始化 ROS 节点, 节点名为 `"test_motor"`。

`ros::NodeHandle`: 创建一个 ROS 节点句柄, 用于与 ROS 主系统通信。

`ros::Rate r(200)`: 设置循环频率为 200Hz, 即每秒执行 200 次。

`livelybot_serial::robot rb`: 创建一个 `robot` 对象, 控制和管理多个电机。

- 获取电机数量并输出日志信息

代码块

```

1  const int motor_num = rb.Motors.size();
2  ROS_INFO("ALL Motor: %d\n", motor_num);
3  ROS_INFO("\033[1;32mSTART\033[0m");

```

`motor_num = rb.Motors.size()`: 获取电机的数量。

`ROS_INFO`: 输出日志信息。

- 控制电机的主循环

代码块

```

1  int cont = 0;
2  float angle = 0.2;
3  while (ros::ok()) // 主循环, 保持程序运行直到节点关闭
4  {
5      ////////////////////////////////////////send
6      for (int i = 0; i < motor_num; i++) // 遍历所有电机
7      {

```

```

8         if(i < (int)motor_num / 2)
9         {
10            rb.Motors[i]->pos_vel_MAXtqe(angle, 0.1, 10); // 对前半部分电机发送控
制指令
11        }
12        else
13        {
14            rb.Motors[i]->pos_vel_MAXtqe(-angle, 0.1, 10); // 对后半部分电机发送
反向控制指令
15        }
16    }
17    cont++;
18    if(cont >= 250) // 每250次反转一次角度的符号
19    {
20        cont = 0;
21        angle *= -1;
22    }
23    rb.motor_send_2(); // 向电机发送命令

```

`while (ros::ok())`: ROS 主循环，只要 ROS 节点没有被关闭，程序会持续运行。

`rb.Motors[i]->pos_vel_MAXtqe(angle, 0.1, 10)`: 向每个电机发送控制指令，控制电机到达指定的位置 `angle`，速度为 `0.1`，最大扭矩为 `10`。前半部分电机控制正向，后半部分电机控制反向。

`angle *= -1`: 每 250 次循环后改变电机目标位置的符号，来实现反转运动。

电机控制主要是依靠这两条指令进行控制

设置控制数据: `rb.Motors[i]->pos_vel_MAXtqe(angle, 0.1, 10);`

发送控制指令: `rb.motor_send_2();`

- 循环控制与等待

代码块

```

1  ros::spinOnce();
2  r.sleep();

```

`ros::spinOnce()`: 在每次循环中处理 ROS 回调队列中的所有回调函数。

`r.sleep()`: 使当前线程等待，直到达到设定的循环频率。

- ROS 终止

代码块

```

1  ros::spin(); // 保持节点运行，直到关闭

```

```
2 return 0;
```

`ros::spin()`: 进入 ROS 主事件循环, 处理消息和回调。程序会一直在这个状态下运行, 直到 ROS 节点被终止。

5.7.2 电机重置零位motor_set_zero

1. 给电机断电再上电。
2. 运行 motor_set_zero.sh 脚本, 等待输出电机状态信息 Motrs[1]: pos 0.001, vel 0.0, tqe 0.0
3. 再给电机断电再上电, 其中所有 pos 项的绝对值小于 0.01 才算通过

六、控制函数

6.1 Motor.cc

6.1.1、构造函数 motor::motor(...)

电机的配置信息:

- 名称、ID、型号、编号
- 位置限制 (pos_upper, pos_lower, pos_limit_enable)
- 扭矩限制 (tor_upper, tor_lower, tor_limit_enable)
- 控制类型 (position, velocity, torque 等)

6.1.2、数据类型转换函数

作用:

- 将浮点制的位置/速度/扭矩值转换为电机所需的 **整数格式 (编码值)**。
- 将数据类型进行转换。将int转float, 和float转int
- 将位置的度/弧度/转数进行转换
- 将速度的度/秒、弧度/秒、转/秒进行转换

6.1.3、控制命令模式函数 (突出)

根据 control_type 调用不同的控制指令函数:

control_type 值	调用函数	模式
1	position()	位置模式
2	velocity()	速度模式
3	torque()	扭矩模式
4	voltage()	电压模式
5	current()	电流模式
6	pos_vel_MAXt qe()	限制扭矩的速 度+位置
9	pos_vel_tqe_k p_kd()	含 KP、KD 的 控制（）速度 积分出来的位 置
10	pos_vel_kp_k d()	含KP、KD的速 度位置控制
11	pos_vel_acc()	含加速度的速 度位置控制
12	pos_vel_tqe_k p_kd2()	运控模式

6.1.4、实际控制函数

下面函数都是向缓冲区写数据，然后进行发送控制

1. 位置模式

说明：

- 使用最大速度和加速度运动到目标位置，并让电机返回状态信息。
- position 目标位置，单位可为转（r）、弧度（rad）、或度（°）。

代码块

```
1 void motor::position(float position)
```

2. 速度模式

说明：

- 以最大加速度加速到指定速度，并让电机返回状态信息。
- velocity 目标速度，单位可为转（rps）、弧度（rad/s）、或度（°/s）。

代码块

```
1 void motor::velocity(float velocity)
```

3. 力矩模式

说明：

- 以给定的力矩转动，并让电机返回状态信息。
- torque 目标力矩，单位牛米（NM）

代码块

```
1 void motor::torque(float torque)
```

4. 电压模式

说明：

- 给Q相电压控制电机，并让电机返回状态信息。
- voltage Q相电压，单位：（V）。

代码块

```
1 void motor::voltage(float voltage)
```

5. 电流模式

说明：

- 给定Q相电流控制电机，并让电机返回状态信息。
- current Q相电流，单位：（A）。

代码块

```
1 void motor::current(float current)
```

6. 位置速度最大力矩模式

说明：

- 以目标速度运动到目标位置，并限制最大输出力矩，并让电机返回状态信息。
- position 目标位置，单位可为转（r）、弧度（rad）、或度（°）。
- velocity 目标速度，单位可为转（rps）、弧度（rad/s）、或度（°/s）。
- torque 目标力矩，单位牛米（NM）。

代码块

```
1 void motor::pos_vel_MAXtqe(float position, float velocity, float torque_max)
```

7. 运控模式

说明：输出力矩 = 位置偏差 * kp + 速度偏差 * kd + tqe。

- position 目标位置，单位可为转（r）、弧度（rad）、或度（°）。
- velocity 目标速度，单位可为转（rps）、弧度（rad/s）、或度（°/s）。
- torque 目标力矩，单位牛米（NM）。

代码块

```
1 void motor::pos_vel_tqe_kp_kd(float position, float velocity, float torque,  
float kp, float kd)
```

8. 速度+位置+kp+kd控制

说明：以给定速度转动到指定位置，并加入pd控制，并让电机返回状态信息。

- position 目标位置，单位可为转（r）、弧度（rad）、或度（°）。
- velocity 目标速度，单位可为转（rps）、弧度（rad/s）、或度（°/s）。

代码块

```
1 void motor::pos_vel_kp_kd(float position, float velocity, float kp, float kd)
```

9. 速度位置加速度控制

说明：指定电机转动到某个位置，并限制转动过程中的最大速度和加速度，并让电机返回状态信息。

- position 目标位置，单位可为转（r）、弧度（rad）、或度（°）。
- velocity 目标速度，单位可为转（rps）、弧度（rad/s）、或度（°/s）。

代码块

```
1 void motor::pos_vel_acc(float position, float velocity, float acc)
```

10. 运控2模式

说明：输出力矩 = 位置偏差 * kp + 速度偏差 * kd + tqe。

- position 目标位置，单位可为转（r）、弧度（rad）、或度（°）。
- velocity 目标速度，单位可为转（rps）、弧度（rad/s）、或度（°/s）。
- torque 目标力矩，单位牛米（NM）。

代码块

```
1 void motor::pos_vel_tqe_kp_kd2(float position, float velocity, float torque, float kp, float kd)
```

11. 停止模式

说明：电机三相都断开，不返回电机状态。

代码块

```
1 void motor::stop()
```

12. 刹车模式

- 说明：电机所有相都短接到地，产生“阻尼刹车”效果，不返回电机状态。

代码块

```
1 void motor::brake()
```

13. 发送查询电机状态指令

说明：仅发送查询电机状态指令，不控制电机

代码块

```
1 void motor::send_state_cmd()
```

14. 设置超时时间

说明：设置电机超时时间ms，重启退出模式

代码块

```
1 void motor::set_motorout(int16_t t_ms)
```

15. 软重启模式

说明：软重启电机状态

代码块

```
1 void motor::reset()
```

6.1.5、状态更新与发布

代码块

```
1 void motor::fresh_data(uint8_t mode, uint8_t fault, int16_t position, int16_t velocity, int16_t torque)
```

- 更新电机当前状态（模式、故障、位置、速度、扭矩）
- 检查是否超过位置或扭矩限制（并报错）

6.1.6、设置和获取电机信息

- `int motor::get_motor_id()`：获取电机id
- `int motor::get_motor_type()`：获取控制模式
- `motor_type motor::get_motor_enum_type()`：获取电机型号
- `std::string motor::get_motor_name()`：获取电机名字

6.2 robot.cc

6.2.1、构造与析构函数

- `robot::robot()`：构造函数，负责参数获取、硬件初始化、线程启动等。
- `robot::~~robot()`：析构函数，负责资源释放、线程回收等。

6.2.2、IMU函数

- `robot::imuCallback`：IMU数据处理函数。
- `robot::imu_limit`：判断IMU姿态是否超限。

6.2.3、关节状态发布

- `robot::publishJointStates`：发布所有电机的关节状态到ROS话题。

6.2.4、电机限位检测

- `robot::detect_motor_limit`：检测电机是否超限。

6.2.5、串口相关操作

- `robot::init_ser`：初始化串口及相关线程。
- `robot::serial_pid_vid`：获取串口PID/VID。
- `robot::list_serial_ports`：枚举串口设备。
- `robot::check_serial_dev_exist`：检测目标串口设备数量。

6.2.6、电机相关操作

- `robot::set_port_motor_num`：设置每个通道的电机数量并获取固件版本。
- `robot::send_get_motor_state_cmd`：发送获取电机状态命令。
- `robot::send_get_motor_version_cmd`：发送获取电机版本命令。
- `robot::motor_send_2`：电机控制指令发送。
- `robot::motor_version_detection`：检测并选择功能版本号。
- `robot::chevk_motor_connection_version`：检测电机连接（版本）。
- `robot::chevk_motor_connection_position`：检测电机连接（位置）。

6.2.7、电机与CAN板控制操作

- `robot::set_stop`：停止所有电机。（是否还要发送函数）
- `robot::set_reset`：重启所有电机。
- `robot::set_reset_zero`：重启回零（全部/指定电机）。
- `robot::set_motor_runzero`：电机运行到零位。
- `robot::set_timeout`：设置电机超时时间。
- `robot::canboard_bootloader`：进入CAN板bootloader模式。
- `robot::canboard_fdcan_reset`：CAN板FDCAN重启。
- `robot::set_data_reset`：数据重置，将所有数据置为 0xFF。

6.2.8、错误检测与恢复

- `robot::check_error`：错误检测与自动恢复线程。

6.3 canboard.cc

6.3.1、构造函数与成员变量初始化

- `canboard::canboard(...)`：初始化 CAN 板对象，创建并初始化所有 CANport。

6.3.2、CANport 相关操作

- `get_CANport()`：获取 CANport 容器的引用。

- `get_CANport_num()`: 获取 CANport 数量。
- `push_CANport()`: 将本对象的所有 CANport 指针添加到外部容器。

6.3.3、电机与CANport控制相关操作

- `motor_send_2()`: 所有 CANport 下发 `motor_send_2` 指令。
- `set_stop()`: 所有 CANport 下发停止指令。
- `set_reset()`: 所有 CANport 下发复位指令。
- `set_reset_zero()`: 所有 CANport 执行重启回零。
- `set_motor_runzero()`: 所有 CANport 执行回零操作。

6.3.4、电机数量与版本相关操作

- `set_port_motor_num()`: 设置每个 CANport 的电机数量，并返回主控板固件版本号。
- `send_get_motor_state_cmd()` / `send_get_motor_state_cmd2()`: 下发获取电机状态指令。
- `send_get_motor_version_cmd()`: 下发获取电机版本指令。
- `set_fun_v()`: 设置功能版本。

6.3.5、数据与参数相关操作

- `set_data_reset()`: 重置所有 CANport 的数据。
- `set_time_out()`: 设置所有或指定 CANport 的超时时间。

6.3.6、CAN板特殊操作

- `canboard_bootloader()`: 让第一个 CANport 进入 bootloader 模式。
- `canboard_fdcan_reset()`: 让第一个 CANport 执行 FDCAN 复位。

6.4 canport.cc

6.4.1、构造与初始化

- `canport::canport(...)`: 构造函数，初始化 CANport 对象、读取参数、创建 motor 对象、建立映射等。

6.4.2、获取电机信息函数

- `set_motor_num()`: 设置电机数量并获取端口固件版本。
- `get_motor_num()`、`get_canboard_id()`、`get_canport_id()`: 获取电机数量、CAN板号、CAN 端口号。

6.4.3、参数设置函数

- `set_conf_load()` / `set_conf_load(int id)`: 恢复参数（全部/指定电机）。

- `set_conf_write()` / `set_conf_write(int id)`: 保存参数（全部/指定电机）。
- `set_reset_zero()` / `set_reset_zero(int id)`: 重置零位（全部/指定电机）。

6.4.4、电机控制函数

- `set_stop()`: 停止所有电机。
- `set_motor_runzero()`: 所有电机回零。
- `set_reset()`: 电机重启。
- `send_get_motor_state_cmd()` / `send_get_motor_state_cmd2()`: 获取电机状态。
- `send_get_motor_version_cmd()`: 获取电机版本。
- `set_fun_v()`: 设置功能版本。
- `set_data_reset()`: 数据区清零。
- `set_time_out()`: 设置电机超时时间。

6.4.5、电机对象管理函数

- `puch_motor()`: 将本端口下所有 motor 指针加入外部 vector。

6.4.6、底层通信函数

- `motor_send_2()`: 通过串口发送当前命令数据。

6.4.7、CAN板特殊操作函数

- `canboard_bootloader()`: 进入 bootloader 模式。
- `canboard_fdcan_reset()`: FDCAN 总线重启。

6.5 lively_serial.cc

6.5.1、构造与析构函数

- `lively_serial::lively_serial(...)`: 构造函数，初始化串口参数并尝试打开串口。
- `lively_serial::~~lively_serial()`: 析构函数，关闭串口并清理标志。

6.5.2、串口数据收发与协议处理函数

- `recv_1for6_42()`: 串口数据接收主循环，解析协议帧、校验、分发处理。
- `send_2()`: 发送数据到串口，计算并填充 CRC8/CRC16 校验，异常处理。

6.5.3、串口状态函数

- `is_serial_error()`: 获取串口错误标志。
- `set_run_flag()` / `get_run_flag()`: 设置/获取串口运行标志。

- `close()`：关闭串口，刷新缓冲区。

6.5.4、协议辅助与外部数据同步

- `port_version_init()`：初始化端口版本号指针。
- `port_motors_id_init()`：初始化电机ID集合和模式标志指针。
- `init_map_motor()`：初始化电机对象映射表指针。
- `port_fun_v_init()`：初始化功能版本号指针。